

Understanding Snort Rules

```
alert tcp any any ->  
any 80 (  
  msg:  
    "HTTP Detect";  
  sid:1000001;  
)
```

Snort Modes

3 Core Modes**01**

Sniffer Mode

Flags: `-v -d -e -i <interface>`

```
$ snort -v -d -e -i eth0
```

Reads IP packets off the network and streams them to the console — headers, payload and data-link layer details.

02

Packet Logger Mode

Flags: `-dev -l <log-dir> -h <network/CIDR>`

```
$ snort -dev -l ./log -h 192.168.1.0/24
```

Captures every inbound and outbound IP packet and writes them to disk, organised in a directory hierarchy by IP address.

03

NIDS / NIPS Mode

Flags: `-d -h <net> -l <log> -c snort.conf`

```
$ snort -d -h 192.168.1.0/24 -l ./log -c snort.conf
```

Most powerful mode: analyses real-time traffic against user-defined rules — generates alerts, logs events, or blocks threats inline.

Anatomy of a Snort Rule

Every Snort rule consists of exactly TWO parts: a Rule Header and Rule Options

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (msg:"ET WEB_SERVER SQL Injection Attempt";  
content:"union"; nocase; sid:2001219; rev:8;)
```

RULE HEADER

Rule Header Components

- Action — alert | log | pass | drop | reject
- Protocol — tcp | udp | icmp | ip
- Source IP — \$EXTERNAL_NET / any / CIDR
- Source Port — any / 80 / !443 / [80,8080]
- Direction — -> (uni) or <> (bi-directional)
- Dest IP & Port — \$HOME_NET / specific

RULE OPTIONS

Rule Option Keywords

- msg — Human-readable alert message
- content — Payload byte pattern match
- pcre — Perl-Compatible Regular Expression
- sid / rev — Unique rule ID / revision
- flags — TCP flags (SYN, ACK, FIN, RST...)
- classtype — Attack classification category

Rule Header: Deep Dive

ACTION

alert | log | pass | drop | reject | sdrops

Defines what Snort does when a match is found. 'alert' generates an alert AND logs. 'drop' (inline mode) silently blocks the packet.

SOURCE IP

\$EXTERNAL_NET | any | 192.168.0.0/24

Supports CIDR notation, negation (!), lists [ip1,ip2], and Snort variables like \$HOME_NET defined in snort.conf.

DIRECTION

-> unidirectional
<> bidirectional

'->' is asymmetric: SRC → DST. Use '<>' for protocols like FTP where data flows both ways (command + data channels).

PROTOCOL

tcp | udp | icmp | ip

Layer 4 protocol filter. 'ip' matches all. Used to scope rule performance — avoid broad protocols unless necessary.

SOURCE PORT

any | 80 | !443 | [80,8080:8090]

Port ranges (80:1024), negation (!80), comma-separated lists. Critically impacts detection engine performance.

DEST IP & PORT

\$HOME_NET | any | specific

Mirror of source fields. Snort variables (\$HTTP_SERVERS, \$SQL_SERVERS) reduce false positives and improve targeting.

Rule Options: Keywords & Categories

GENERAL

msg:

Human-readable label: msg:"Malware Beacon"

sid:

Unique signature ID. <100 = reserved; 100-999999 = Snort; 1000000+ = local rules

rev:

Rule revision number. Always increment when modifying an existing rule

classtype:

Pre-defined attack category (trojan-activity, web-application-attack, etc.)

DETECTION

content:

Payload byte pattern: content:"GET /malware.exe". Supports | hex | encoding

pcre:

Regex engine: pcre:"/passwd/i". Most powerful but CPU-expensive

nocase

Case-insensitive content matching modifier

distance:/within:

Relative anchoring of multiple content patterns in sequence

FLOW / STATE

flow:

TCP stream direction: to_server,established — avoids matching handshake noise

flags:

TCP flag inspection: flags:S (SYN scan), flags:SF (stealth FIN scan)

dsize:

Payload size filter: dsize:0; matches empty payloads (C2 heartbeats)

byte_test:

Binary field comparison — critical for protocol-level malware analysis

THRESHOLD & RESPONSE

threshold:

Rate-limits alerts: type limit,track by_src,count 5,seconds 60

priority:

Alert severity 1 (highest) to 10 (lowest) — feeds SIEM triage logic

metadata:

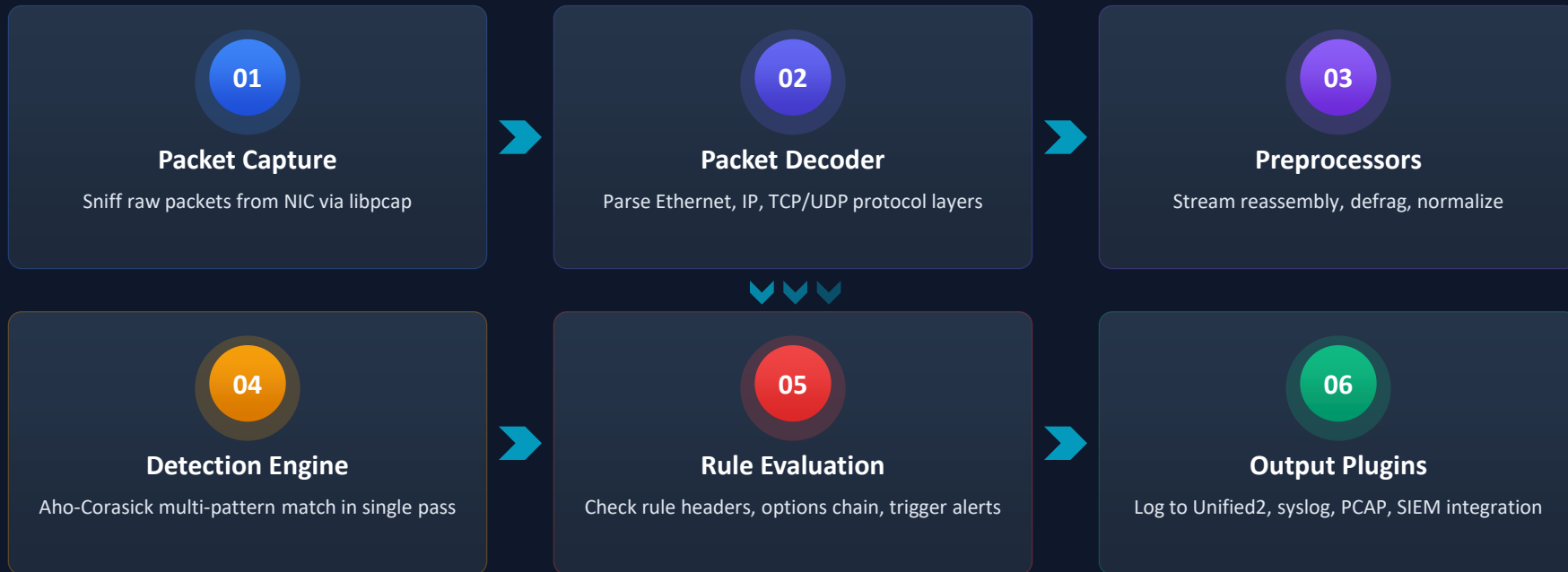
Attaches MITRE ATT&CK IDs, CVE references, affected_product info

reference:

Links to CVE/BugTraq/Snort SID DB: reference:cve,2021-44228

Snort Processing Pipeline

How Snort processes network traffic from capture to alert



Network Traffic ▶ Capture ▶ Decode ▶ Process ▶ Detect ▶ Evaluate ▶ Alert / Log

Practical Rule Writing: Case Study

[⚠ WannaCry / MS17-010](#)

Scenario: Detect WannaCry SMB Exploit (EternalBlue / MS17-010) lateral movement over port 445

SNORT RULE

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 445
(msg:"ET EXPLOIT MS17-010 WannaCry SMB";
flow:to_server,established; content:"|FF|SMB"; depth:5;
pcr:"/\x00Trans2/"; metadata:cve 2017-0144; classtype:attempted-admin; sid:2024217; rev:5;)
```

RULE BREAKDOWN

Flow State Filter

```
flow:to_server,established
```

Only inspect established TCP sessions going TO port 445. Eliminates handshake noise — reduces false positives dramatically.

SMB Magic Bytes

```
content:"|FF|SMB"; depth:5
```

Hex pattern matching for SMB signature. |FF| is 0xFF. depth:5 limits search to first 5 bytes of payload for performance.

PCRE for Trans2 Request

```
pcr:"/\x00Trans2/"
```

Regex detects the Trans2 sub-command byte used by EternalBlue. Runs AFTER content pre-filter — prevents engine overhead.

MITRE / CVE Linkage

```
metadata:cve 2017-0144
```

Attaches CVE-2017-0144 to the alert. Enables automated SIEM correlation in Splunk, Elastic, and MISP threat-intel platforms.

Advanced Rule Features

byte_test — Binary Protocol Inspection

```
byte_test:4,>,0x1000,0,relative;
```

Extracts N bytes from a payload offset and compares as integer. Essential for detecting malformed protocol headers in C2 channels and buffer overflows.

isdataat — Payload Existence Check

```
isdataat:512,relative;
```

Tests whether data exists at a byte offset. Detects unusually long fields indicating buffer overflow attempts. Pairs with content + distance for chained detection.

urilen — URL Length Anomaly Detection

```
urilen:>500;
```

Fires when URL length exceeds a threshold. Detects SQL injection (UNION SELECT chains), XSS payloads, and path traversal attacks against web apps.

PCRE Cheat Sheet

- ▶ `/cmd\.exe/i` – case-insensitive
- ▶ `/[\x41-\x5A]{50,}/` – long caps
- ▶ `/(?:UNION|SELECT)/i` – SQL injection
- ▶ `/^GET\[^\]/{200,}/` – long URI
- ▶ `/\x4D\x5A.{0,512}\x50\x45/s` – PE header

Threshold Tuning

`threshold: type both, track by_src, count 10, seconds 60;`

Fire alert once per src IP per minute only after 10 matches — prevents brute-force alert floods while catching scans.

Snort in Digital Forensics & Malware Analysis

Forensic Applications

Network Traffic Reconstruction

Unified2 log format feeds Wireshark, tcpdump, and NetworkMiner. Log all packets related to an alert for full session replay. Critical for post-breach timeline reconstruction.

C2 Beacon Detection

Periodic outbound connections with consistent timing intervals (dsize:0, flow:established,to_server) fingerprint malware heartbeats — Cobalt Strike, Metasploit, and RATs use predictable patterns.

Data Exfiltration Indicators

Monitor anomalous outbound volumes using byte counters. Rules targeting DNS over TCP, unusually large ICMP payloads, or FTP transfers to unknown external IPs.

Lateral Movement Detection

Cross-host SMB/RDP/WMI anomalies. Writing Snort rules that trigger on internal traffic (HOME_NET → HOME_NET) catches worm propagation and pass-the-hash attacks.

Malware Family → Snort Signature Mapping

WannaCry

SMB/445

EternalBlue exploit byte pattern matching in Trans2 SMB sub-commands

Mirai Botnet

Telnet/23

Brute-force login strings + 'busybox' payload; threshold on failed auths

Emotet

HTTP/443

Malformed User-Agent strings + encoded POST body patterns; PCRE on binary blobs

Cobalt Strike

HTTP/S C2

Default malleable C2 profile headers; Beacon URI regex patterns match /jquery-3.3.1.min.js

Dridex

HTTPS/P2P

DGA domain characteristics; binary format of HTTPS POST payloads with byte_test

BlackMatter

SMB/RDP

Encrypted channel detection via dsize anomalies + flow state + timing threshold rules

Rule Writing Methodology & Best Practices

01 Be Specific, Not Broad

Use flow:, proto, and dsize filters before content/pcre. Broad rules create alert fatigue and miss real threats hidden in the noise.

02 Use Snort Variables

\$HOME_NET, \$EXTERNAL_NET, \$HTTP_SERVERS — always prefer variables over hardcoded IPs. They adapt to environment changes without rule rewrites.

03 Anchor Your Patterns

Always use depth:, offset:, distance:, within: to constrain where pattern matching occurs. Unanchored content rules are slow and generate false positives.

04 Test Before Deploying

Use tcpreplay to replay PCAP files against your rules in test mode (snort -A console -r test.pcap). Never push untested rules to production IPS.

05 Leverage Talos Intelligence

Cisco Talos (ET Open rules, Snort community rules) provides production-quality signatures. Study their methodology — read SIDs 2000000+ for real-world patterns.

06 Document & Version Control

Always increment rev:, use metadata: for CVE/MITRE ATT&CK tags, and maintain rules in Git. Correlate rule changes to threat intelligence updates.

LAB

IDS Lab — System Architecture Overview

🌐 Internal Network — 192.168.1.0/24

🐱 KALI LINUX

📍 IP: 192.168.1.106

🔴 ROLE: Attacker

🔧 sqlmap v1.9.9.4

Automated SQL Injection Tool

- --dbs (enumerate databases)
- --batch (auto mode)
- --tables / --dump

📄 Attack Target
<http://192.168.1.107/mutillidae/index.php?username=&password=>

👤 THREAT ACTOR

🐧 UBUNTU — IDS MONITOR

📍 IP: 192.168.1.109 | Interface: ens33

🔵 ROLE: Passive Network Monitor / IDS

📄 Snort 2 IDS

Mode: Promiscuous (Passive)

Command: `sudo snort -q -c /etc/snort/snort.conf -i ens33 -A console`

Rule Sources:

- PulledPork (ET Community Rules)
- Custom SQLi Rules (SID 1000050-1000057)

🔴 SQLi Attack | `sqlmap --dbs --batch`

📄 IDS_Setup.sh
Installs PulledPork + configures Snort for subnet 192.168.1.0/24

✅ DEFENDER / MONITOR

🐙 METASPLOITABLE 2

📍 IP: 192.168.1.107

🔴 ROLE: Vulnerable Target VM

🌐 Apache Web Server
Port: 80
Serving vulnerable web apps

💀 Mutillidae App
`/mutillidae/index.php`

- SQL Injection
- phpMyAdmin / DVWA / TWiki

⚠️ VICTIM / TARGET

LAB Requirements



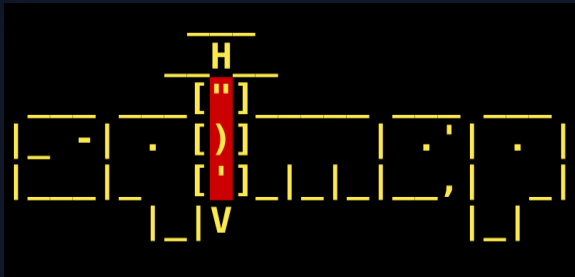
Kali



Ubuntu



Metasploitable 2



IDS_Setup.sh

```
#!/  
/bin/bash  
$ echo "
```

```
black@black: ~/Desktop  
black@black:~/Desktop$ sudo ./IDS_setup.sh  
[*] Initializing Black Lab Snort 2 Setup...  
[*] Installing PulledPork dependencies...  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
libcrypt-ssleay-perl is already the newest version (0.73.06-2build5).  
liblwip-useragent-determined-perl is already the newest version (1.07-3).  
0 upgraded, 0 newly installed, 0 to remove and 222 not upgraded.  
[*] Installing PulledPork...  
[*] Setting ens33 to promiscuous mode...  
[*] Creating required directories...  
[*] Writing pulledpork.conf...  
[*] Configuring snort.conf for 192.168.1.0/24...  
[*] Running PulledPork to download and compile rules...
```

```
Done  
Please review /var/log/sid_changes.log for additional details  
Fly Piggy Fly!
```

```
[+] =====  
[+] Setup Complete. Snort 2 rules are ready for ens33.  
[+] Start the IDS by running this exact command:
```

```
sudo snort -q -c /etc/snort/snort.conf -i ens33 -A console
```

```
[+] =====  
black@black:~/Desktop$
```

```
black@black: ~/Desktop  
black@black:~/Desktop$ sudo snort -q -c /etc/snort/snort.conf -i ens33 -A console
```

IP: 192.168.1.109



Ubuntu

IP: 192.168.1.106



Kali

```
(black@black)-[~/Desktop]
```

```
$ sqlmap -hh
```

```
{1.9.9.4#dev}
```

```
https://sqlmap.org
```

Usage: python sqlmap [options]

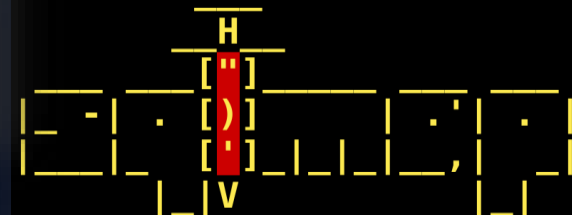
Options:

-h, --help	Show basic help message and exit
-hh	Show advanced help message and exit
--version	Show program's version number and exit
-v VERBOSE	Verbosity level: 0-6 (default 1)

Target:

At least one of these options has to be provided to define the target(s)

-u URL, --url=URL	Target URL (e.g. "http://www.site.com/vuln.php?id=1")
-d DIRECT	Connection string for direct database connection
-l LOGFILE	Parse target(s) from Burp or WebScarab proxy log file



IP: 192.168.1.107

Web Server Is Running

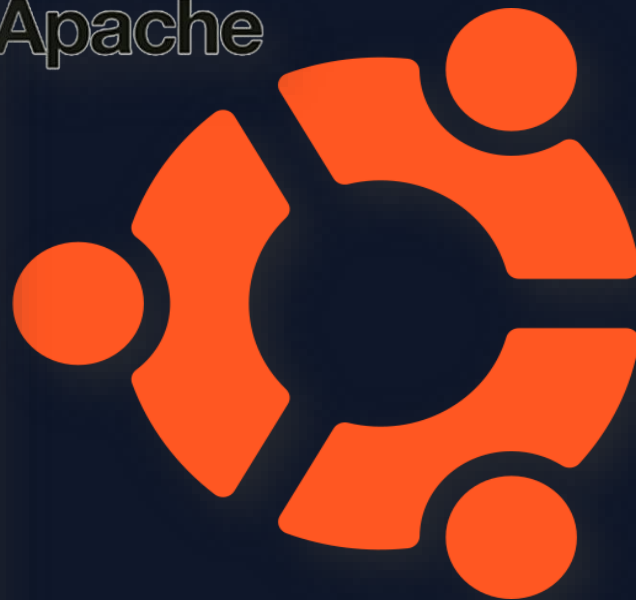
metasploitable

Warning: Never expose this VM to an untrusted network!

Contact: msfdev[at]metasploit.com

Login with msfadmin/msfadmin to get started

- [TWiki](#)
- [phpMyAdmin](#)
- [Mutillidae](#)
- [DVWA](#)
- [WebDAV](#)



Metasploitable 2



IP: 192.168.1.106

Kali

STARTING THE ATTACK

```
(black@black)-[~/Desktop]
$ sqlmap -u 'http://192.168.1.107/mutillidae/index.php?page=user-info.php&username=&password=&user-info-php-submit-button=View+Account+Details' --batch --dbs
```



[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 14:58:22 /2026-02-21/

[14:58:22] [WARNING] provided value for parameter 'username' is empty. Please, always use only valid parameter values so sqlmap could be able to run properly

[14:58:22] [WARNING] provided value for parameter 'password' is empty. Please, always use only valid parameter values so sqlmap could be able to run properly

[14:58:22] [INFO] testing connection to the target URL

you have not declared cookie(s), while server wants to set its own ('PHPSESSID=7ffdb75bd87...ba28ff6df0'). Do you want to use those [Y/n] Y

[14:58:22] [INFO] checking if the target is protected by some kind of WAF/IPS

[14:58:22] [INFO] testing if the target URL content is stable



Ubuntu

IP: 192.168.1.109

Snort IDS detected the sql injection payloads

```
black@black:~/Desktop$ sudo snort -q -c /etc/snort/snort.conf -i ens33 -A console
02/21-14:58:17.285377  [**] [1:2006446:15] ET WEB_SERVER Possible SQL Injection Attempt UNION SELECT in HTTP URI [**]
[Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.106:46366 -> 192.168.1.107:80
02/21-14:58:17.285377  [**] [1:2017808:1] ET WEB_SERVER Possible MySQL SQLi Attempt Information Schema Access [**] [
[Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.106:46366 -> 192.168.1.107:80
02/21-14:58:17.285377  [**] [1:2049400:1] ET WEB_SERVER /etc/passwd Detected in URI [**] [Classification: Attempted
Information Leak] [Priority: 2] {TCP} 192.168.1.106:46366 -> 192.168.1.107:80
02/21-14:58:17.285377  [**] [1:2009815:5] ET WEB_SERVER Attempt To Access MSSQL xp_cmdshell Stored Procedure Via URI
[**] [Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.106:46366 -> 192.168.1.107:80
02/21-14:58:17.285377  [**] [1:2009714:9] ET WEB_SERVER Script tag in URI Possible Cross Site Scripting Attempt [**]
[Classification: Web Application Attack] [Priority: 1] {TCP} 192.168.1.106:46366 -> 192.168.1.107:80
02/21-14:58:17.809460  [**] [1:2008538:8] ET SCAN Sqlmap SQL Injection Scan [**] [Classification: Attempted Informat
ion Leak] [Priority: 2] {TCP} 192.168.1.106:46398 -> 192.168.1.107:80
02/21-14:58:17.940218  [**] [1:2008538:8] ET SCAN Sqlmap SQL Injection Scan [**] [Classification: Attempted Informat
ion Leak] [Priority: 2] {TCP} 192.168.1.106:46408 -> 192.168.1.107:80
02/21-14:58:18.091610  [**] [1:2008538:8] ET SCAN Sqlmap SQL Injection Scan [**] [Classification: Attempted Informat
ion Leak] [Priority: 2] {TCP} 192.168.1.106:46416 -> 192.168.1.107:80
02/21-14:58:18.226856  [**] [1:2008538:8] ET SCAN Sqlmap SQL Injection Scan [**] [Classification: Attempted Informat
ion Leak] [Priority: 2] {TCP} 192.168.1.106:46428 -> 192.168.1.107:80
02/21-14:58:18.363754  [**] [1:2008538:8] ET SCAN Sqlmap SQL Injection Scan [**] [Classification: Attempted Informat
ion Leak] [Priority: 2] {TCP} 192.168.1.106:46438 -> 192.168.1.107:80
02/21-14:58:18.500910  [**] [1:2008538:8] ET SCAN Sqlmap SQL Injection Scan [**] [Classification: Attempted Informat
ion Leak] [Priority: 2] {TCP} 192.168.1.106:46450 -> 192.168.1.107:80
02/21-14:58:18.533035  [**] [1:2008538:8] ET SCAN Sqlmap SQL Injection Scan [**] [Classification: Attempted Informat
ion Leak] [Priority: 2] {TCP} 192.168.1.106:46454 -> 192.168.1.107:80
```



SQL Injection Attack Flow — Detected by Snort IDS

✖ KALI | 192.168.1.106

🛠 METASPLOITABLE | 192.168.1.107

🛡 SNORT IDS | 192.168.1.109

PHASE 1 — Tool Fingerprint & Initial Probe

sqlmap --dbs --batch
User-Agent: sqlmap/1.9.9.4

HTTP GET /mutillidae/index.php?username=&password=

HTTP 200 OK
+ Set PHPSESSID cookie

🚨 SID:1000050 FIRED
SQLi - sqlmap User-Agent Detected
[Priority: 1] Web-App-Attack

PHASE 2 — Boolean & Error-Based Injection

Payload: ' OR '1' --
Boolean logic injection

?username' OR '1' --

DB Error OR Auth Bypass
Returns user data

🚨 SID:1000052 FIRED
SQLi - Boolean Logic (OR 1=1)
🚨 SID:1000053 FIRED
SQLi - Quote Breakout + Comment

PHASE 3 — UNION-Based Data Exfiltration

1 UNION SELECT table_name
FROM information_schema.tables--

UNION SELECT payload in URI

DB tables exposed!
Returns schema data

🚨 SID:1000054 FIRED
SQLi - UNION SELECT Execution
🚨 SID:1000055 FIRED
SQLi - info_schema Target

PHASE 4 — Time-Based Blind SQLi

' AND SLEEP(5)--
benchmark(1000000,MD5(1))

Time-delay payload in URI

Delayed Response (~5s)
Confirms blind SQLi vector

🚨 SID:1000056 FIRED
SQLi - Time-Based Blind (SLEEP)
🚨 SID:1000057 FIRED
SQLi - Time-Based Blind (BENCHMARK)

PHASE 5 — ET Community Rules Triggered (Snort built-in)

🚨 ET WEB_SERVER Possible SQL Injection Attempt UNION SELECT in HTTP URI [SID:2006446] Priority: 1 TCP 192.168.1.106 → 192.168.1.107:80
🚨 ET WEB_SERVER Possible MySQL SQLi Attempt Information Schema Access [SID:2017808] Priority: 1
🚨 ET WEB_SERVER /etc/passwd Detected in URI [SID:2049400] Priority: 2 | Attempted Information Leak
🚨 ET SCAN Sqlmap SQL Injection Scan [SID:2008538] Priority: 2 * Multiple TCP Connections (port 46366 → 46454)

✅ Total Custom Rules Fired: SID 1000050 – 1000057 | ✅ ET Community Rules Fired: SID 2006446, 2017808, 2049400, 2008538 | 🛡 All attacks detected in real-time by Snort IDS

RULE SET 1 Detecting the Automated Tool (sqlmap)

1 Catching the Default sqlmap User-Agent

SID: 1000050

```
alert tcp 192.168.1.102 any -> 192.168.1.107 80 (  
  msg:"SQLi - sqlmap User-Agent Detected";  
  flow:to_server,established; content:"User-Agent|3A|"; http_header;  
  content:"sqlmap"; nocase; http_header; classtype:web-application-attack;  
  sid:1000050; rev:1;)
```

► RULE BREAKDOWN

ACTION

alert — triggers an IDS alert event

FLOW

to_server, established

DETECT

User-Agent header contains sqlmap keyword

CLASSIFY

web-application-attack | sid:1000050 | rev:1

RULE SET 1 Detecting the Automated Tool (sqlmap)

2 Catching sqlmap Dummy Integer Testing e.g., ?id=1234 AND 4321=4321

SID: 1000051

```
alert tcp 192.168.1.102 any -> 192.168.1.107 80 (  
  msg:"SQLi - sqlmap Dummy Integer Injection";  
  flow:to_server,established;  
  pcre:"/?.*=d{4,5}s+(AND|OR)s+d{4,5}=d{4,5}/i"; http_uri;  
  classtype:web-application-attack; sid:1000051; rev:1;)
```

► RULE BREAKDOWN

ACTION

alert — triggers an IDS alert event

FLOW

to_server, established

DETECT

pcre: ?param=NNNN AND NNNN=NNNN in URI (http_uri)

CLASSIFY

web-application-attack | sid:1000051 | rev:1

RULE SET 2 Boolean & Error-Based Logic Detection

1 Classic Boolean OR Logic catches spaces, ticks & URL encoding

SID: 1000052

```
alert tcp 192.168.1.102 any -> 192.168.1.107 80 (  
  msg:"SQLi - Boolean Logic (OR 1=1)";  
  flow:to_server,established;  
  pcre:"/(%27|')(s|%20)*(OR|AND)(s|%20)*d*(s|%20)*=(s|%20)*d*/i"; http_uri;  
  classtype:web-application-attack; sid:1000052; rev:1;)
```

► RULE BREAKDOWN

ACTION

alert — triggers an IDS alert event

FLOW

to_server, established

DETECT

pcre: quote + OR/AND + numeric equality (URL-encoded, http_uri)

CLASSIFY

web-application-attack | sid:1000052 | rev:1

RULE SET 2 Boolean & Error-Based Logic Detection

2 Quote Breakout & Comment Injection single quotes followed by -- or

SID: 1000053

```
alert tcp 192.168.1.102 any -> 192.168.1.107 80 (  
  msg:"SQLi - Quote Breakout and Comment";  
  flow:to_server,established;  
  pcre:"/(%27|')s*(%23|--|%2D%2D)/i"; http_uri;  
  classtype:web-application-attack; sid:1000053; rev:1;)
```

► RULE BREAKDOWN

ACTION

alert — triggers an IDS alert event

FLOW

to_server, established

DETECT

pcre: single-quote followed by -- or # comment tokens

CLASSIFY

web-application-attack | sid:1000053 | rev:1

RULE SET 3 Data Exfiltration (UNION-Based)

1 UNION SELECT Execution in URI

SID: 1000054

```
alert tcp 192.168.1.102 any -> 192.168.1.107 80 (  
  msg:"SQLi - UNION SELECT Execution";  
  flow:to_server,established;  
  content:"union"; nocase; http_uri; content:"select";    nocase; http_uri;  
  distance:0;  
  classtype:web-application-attack; sid:1000054; rev:1;)
```

► RULE BREAKDOWN

ACTION

alert — triggers an IDS alert event

FLOW

to_server, established

DETECT

content: "union" then "select" chained in URI (nocase)

CLASSIFY

web-application-attack | sid:1000054 | rev:1

RULE SET 3 Data Exfiltration (UNION-Based)

2 Database Structure Mapping targeting information_schema

SID: 1000055

```
alert tcp 192.168.1.102 any -> 192.168.1.107 80 (  
  msg:"SQLi - info_schema Target";  
  flow:to_server,established;  
  content:"information_schema"; nocase; http_uri;  
  classtype:web-application-attack; sid:1000055; rev:1;)
```

► RULE BREAKDOWN

ACTION

alert — triggers an IDS alert event

FLOW

to_server, established

DETECT

content: "information_schema" in URI (nocase)

CLASSIFY

web-application-attack | sid:1000055 | rev:1

RULE SET 4 Time-Based Blind SQLi Detection

1 MySQL SLEEP() Command Detection

SID: 1000056

```
alert tcp 192.168.1.102 any -> 192.168.1.107 80 (  
  msg:"SQLi - Time-Based Blind (SLEEP)";  
  flow:to_server,established;  
  pcre:"/(%27|').*sleep(s|%20)*(/i"; http_uri;  
  classtype:web-application-attack; sid:1000056; rev:1;)
```

► RULE BREAKDOWN

ACTION

alert — triggers an IDS alert event

FLOW

to_server, established

DETECT

pcre: quote followed by sleep() function call (http_uri)

CLASSIFY

web-application-attack | sid:1000056 | rev:1

RULE SET 4 Time-Based Blind SQLi Detection

2 MySQL BENCHMARK() Command Detection

SID: 1000057

```
alert tcp 192.168.1.102 any -> 192.168.1.107 80 (  
  msg:"SQLi - Time-Based Blind (BENCHMARK)";  
  flow:to_server,established;  
  pcre:"/benchmark(s|%20)*(/i"; http_uri;  
  classtype:web-application-attack; sid:1000057; rev:1;)
```

► RULE BREAKDOWN

ACTION

alert — triggers an IDS alert event

FLOW

to_server, established

DETECT

pcre: benchmark() function call in URI (URL-encoded)

CLASSIFY

web-application-attack | sid:1000057 | rev:1

PROMPT:

You are a Digital Forensics and malware analyst expert. your work is to give slides about: "Understanding Snort Rules: Structure, Syntax, and Academic Logic Behind Signatures."

Based on your experience and knowledge, give a good subject in 10 slides. Give the core concept related to the info here and also your experience in the malware & digital forensics fields



RESOURCES:

Kali Linux: <https://www.kali.org/get-kali/>

Ubuntu: <http://ubuntu.com/download>

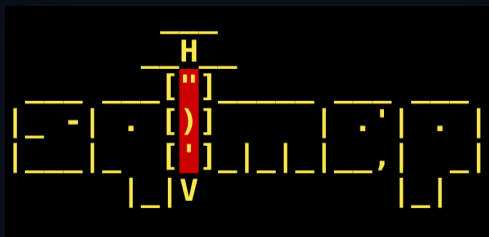
Virtual Box: <http://virtualbox.org/wiki/Downloads>

Snort 2 : <https://www.snort.org/snort2>

SQLmap: <https://github.com/sqlmapproject/sqlmap>



THANK YOU



Apache